



1



2



3



4

ENGINEERING HANDBOOK · 2026

Async Operations in Agentic AI Systems

A Practitioner's Guide for Databricks & Production AI Teams — patterns, pitfalls, and the concurrency plumbing that actually holds up under load.



Sourav Banerjee — Resident Solution Architect
September 2026 · First Edition

DATABRICKS · FIELD ENGINEERING

Async Operations in Agentic AI Systems — A Practitioner's Guide

First Edition · September 2026.

This handbook is written for senior AI/ML engineers, platform architects, and Databricks Resident Solution Architects who ship agents under production SLDs. Everything here is field material: patterns that survived contact with rate limits, streaming clients, and the on-call rotation.

Code samples target Python 3.11+. Framework snippets reference the current stable APIs of OpenAI, Anthropic, LangChain / LangGraph, MLflow and the Databricks Vector Search / Model Serving stack. Where the API surface has multiple valid forms, we chose the one that fails loudest under load.

The trademarks, product names and logos referenced herein are the property of their respective owners.

Contents

CH 01	Why Async Matters in Agentic AI	4
CH 02	The asyncio Mental Model	7
CH 03	The Four Async Patterns in an Agent Loop	10
3.1	Parallel Tool Fan-out	10
3.2	Streaming Token Generation	11
3.3	Concurrent Multi-Agent Orchestration	12
3.4	Background & Fire-and-Forget	13

CH 04	Concurrency Control — The Part Everyone Gets Wrong	15
CH 05	Sync vs Async in Real Frameworks	18
CH 06	Common Failure Modes with Fixes	22
CH 07	Production Architecture on Databricks	25
CH 08	Checklist — Is My Agent Truly Async?	28
REFS	References & Further Reading	30

CHAPTER ONE

Why Async Matters in Agentic AI

Agents don't do work — they wait. The whole game is deciding *what* to wait on next, and never blocking the loop while you do it.

Modern agent workloads are, almost without exception, I/O-bound. An LLM call to a hosted endpoint takes anywhere from two seconds for a tight completion to thirty seconds for a long-context reasoning pass. Retrieval hits a vector store over HTTP. Tool calls resolve to REST APIs, SQL warehouses, or user-defined Unity Catalog functions. Guardrails and moderation add another network hop. If you profile a real production agent you will find the CPU dozing at low single-digit utilization while the process spends 96–99% of wall time in `recv()`. That is not a bug — that is the shape of the workload — and it is precisely why `asyncio` is not a nice-to-have but the substrate on which the entire system has to sit.

The failure mode that gets teams into trouble is what we'll call the **sync agent latency trap**. Consider a perfectly ordinary five-step ReAct agent: the model plans, calls a tool, observes the result, plans again, calls another tool, and produces a final answer. Each round trip — model plus tool plus post-processing — is around four seconds. If the code is written synchronously, those five steps stack: $4s + 4s + 4s + 4s + 4s = 20$ **seconds of wall time**. The user staring at a blinking cursor perceives a broken product long before the twenty seconds are up. Worse, the process handling that request cannot serve any other user during those twenty seconds — so a single sync agent behind an HTTP endpoint runs out of worker capacity at trivial concurrency.

Async changes the arithmetic. A single event loop can hold hundreds of in-flight `await`s simultaneously; while any one of them is parked waiting on I/O, the loop is free to advance others. When two calls in a step are independent — say, a retrieval and a web search — the wall time for that step collapses toward `max(step)` instead of `sum(step)`. Across an agent that runs many such steps and serves many concurrent users, the effect compounds: the same host that could barely serve five simultaneous users synchronously will comfortably serve several hundred asynchronously, because it isn't holding a whole OS thread idle for each one.

Sync execution — 4 tool calls, 4s each



Async execution — same 4 calls, gathered



FIGURE 1.1 Four independent tool calls run sequentially versus overlapped. Async does not make any single call faster — it removes the dead time between them.

WHY THIS MATTERS

Perceived latency, cost per resolved query, and throughput per GPU-hour all share the same lever: how many calls you can have in flight simultaneously per host. Sync agents burn worker slots. Async agents burn network sockets. Sockets are cheap; worker slots are not.

Production drivers

Four numbers usually determine whether an agent is production-ready, and every one of them is sensitive to concurrency. **Throughput per GPU-hour** falls when serving hosts idle behind synchronous callers, because the model endpoint is starved of requests. **Cost per resolved query** rises when retries and long-running steps hold onto worker processes. **p95 latency SLOs** — the ones your SRE will actually alert on — are dominated by the tail behavior of the slowest step in an agent chain; async lets you overlap that step with everything else and stops one slow retrieval from dragging the whole request into the alert threshold. Finally, **streaming UX** is the free win most teams leave on the table: the total tokens generated are identical, but a user who sees the first token in 400ms perceives the system as being an order of magnitude snappier than one who waits eight seconds for a completed block.

DATABRICKS / PRODUCTION NOTE

On Databricks Model Serving the endpoint is provisioned with a fixed concurrency budget — requests over that ceiling are queued. If your agent host is synchronous, one slow serving call blocks the caller for its full duration and the queue upstream grows faster than it drains. An async client with proper timeouts lets you fan out to Model Serving, Vector Search, and UC Functions on the same request without paying that queue-tax, and lets you actually saturate the endpoint's provisioned concurrency instead of throttling yourself in the client.

CHAPTER TWO

The asyncio Mental Model

A short, opinionated refresher on the three primitives you cannot bluff your way past: the loop, the coroutine, and the yield point.

The **event loop** is the scheduler at the heart of every asyncio program. There is exactly one running per thread, and its job is boring but critical: pick a ready coroutine off a queue, run it until it hits an `await`, park it, and pick another. There is no preemption. Coroutines cooperate; the loop merely enforces turns. If you hold the loop — by running a tight CPU loop, or by calling a synchronous library that blocks on a socket — every other coroutine in your process stops, including the one that reads the next HTTP request.

A **coroutine** is a suspendable function: any `async def` creates one, and calling it returns a coroutine object rather than executing the body. You feed that object to the loop either directly with `await` (which schedules it and waits for the result) or by wrapping it in a `Task` via `asyncio.create_task`, which schedules it immediately and returns a handle you can cancel, inspect, or await later. A **Future** is the underlying placeholder for a not-yet-available result — Tasks are Futures wrapped around a coroutine. In practice you almost always work with Tasks; Futures come up mainly when you're bridging to callback-based APIs.

Everything else is combinators over those three. `asyncio.gather` takes N coroutines and returns their results together; `asyncio.as_completed` yields them one at a time as each finishes; `asyncio.wait` gives you fine-grained control with `FIRST_COMPLETED` / `ALL_COMPLETED` semantics. The Python 3.11 `TaskGroup` replaces some raw `gather` usage with structured concurrency guarantees, which we come back to in Chapter 4.

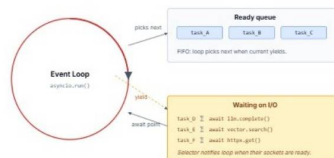
PYTHON 3.11 - THE PRIMITIVES, IN TEN LINES

```
import asyncio, httpx

async def fetch(url: str) -> str:
    async with httpx.AsyncClient() as c:
        r = await c.get(url, timeout=10) # await = yield point
        return r.text

async def main():
    urls = ["https://a", "https://b", "https://c"]
    results = await asyncio.gather(*(fetch(u) for u in urls))
    return results

asyncio.run(main())
```



Coroutines flow between Ready → Waiting; the loop advances only when the current task yields.

FIGURE 2.1 The event loop as a two-queue scheduler. Every `await` is a hand-off; blocking code steals the whole loop.

Cooperative multitasking, in one rule

Because asyncio is cooperative, an `await` is the only place execution can be handed to another task. Any function you call that does *not* yield — `requests.get(...)`, `time.sleep(1)`, a heavy JSON parse over a megabyte payload, a sync database driver — holds the loop for its entire duration. This is the single most common reason an “async” agent behaves synchronously in production: someone imported the sync client out of habit.

The rule to internalize is simple: **everything on the hot path is non-blocking, or it is offloaded.** Non-blocking means an async-native client (`httpx.AsyncClient`, `aiomysql`, `AsyncOpenAI`, `databricks.vector_search.async_client`). Offloaded means you wrap the sync call so it runs on a thread and the loop stays free.

PYTHON 3.11 - OFFLOADING A BLOCKING LIBRARY

```
import asyncio, requests # requests is sync - will block the loop

def blocking_lookup(syn: str) -> dict:
    return requests.get(f"https://api.example.com/q/{syn}").json()

async def lookup(syn: str) -> dict:
    # run in executor / to thread offloads to a threadpool; loop stays free
    return await asyncio.to_thread(blocking_lookup, syn)
```

A GOTCHA

`asyncio.to_thread` uses the default thread pool, which is capped (typically 32 workers or so depending on Python version). It is a bridge, not a scalability strategy — if a “hot path” call ends up wrapped in `to_thread`, you’re back to thread-per-request semantics for that leg. Prefer an async-native driver; use `to_thread` only for the last-mile sync library you can’t replace.

CHAPTER THREE

The Four Async Patterns in an Agent Loop

Almost every agent you'll ever build reduces to these four moves. Learn them once; recognize them everywhere.

Once you internalize the loop-and-await mental model, the interesting question becomes structural: how should the agent use concurrency? In production agents there are four recurring shapes. Get comfortable naming them, because most design reviews on agent systems degenerate into arguments that turn out to be about which of these patterns you're actually applying.

3.1 Parallel Tool Fan-out

The classic case: the model produces a plan whose next step calls several independent tools. A support agent needs to enrich a ticket by querying the CRM, the KB vector index, and a live status page all at once. If the tools do not depend on each other, `asyncio.gather` — or better, a `TaskGroup` — is the right primitive. Wall time collapses to the slowest tool. Use fan-out any time the model's planning output is a set of tool invocations rather than an ordered sequence.

PYTHON 3.11: FAN-OUT WITH TASKGROUP

```
import asyncio

async def enrich(ticket_id: str) -> dict:
    async with asyncio.TaskGroup() as tg:
        t_web = tg.create_task(web_search(ticket_id))
        t_vec = tg.create_task(vector_search(ticket_id))
        t_sql = tg.create_task(sql_query(ticket_id))
    return {
        "web": t_web.result(),
        "vector": t_vec.result(),
        "sql": t_sql.result(),
    }
```



FIGURE 3.1 Fan-out. The wall time of the whole step equals the slowest branch — provided the branches are truly independent.

3.2 Streaming Token Generation

Streaming is where the perceived-latency win lives. Instead of awaiting a complete response, the client iterates over an async generator that yields tokens as the model produces them. In an agent this gets more interesting than in a plain chatbot, because the model may emit tool-call tokens mid-stream. The pattern is: buffer the deltas, watch for a function-call marker in the parsed event, if found stop forwarding tokens to the user, run the tool, feed the result back into a follow-up completion, and resume streaming from that new stream. The user sees a continuous ribbon of text; the orchestrator quietly rewires the pipe.



FIGURE 3.2 The orchestrator sits between model and user, forwarding text tokens straight through but siphoning tool-call deltas into a side channel.

3.3 Concurrent Multi-Agent Orchestration

When the design escalates from “one agent with tools” to “a supervisor coordinating specialists,” the same fan-out pattern applies but the branches are themselves agents. A research supervisor might dispatch a market-analyst, a competitive-scan and a financials specialist in parallel; the sequential handoff variant would run them one after another and wait days on wall time equivalents. Parallel dispatch is safe when the specialists have *independent* subtasks and no shared writable state. It is unsafe — and quietly produces wrong answers — when their outputs depend on each other’s intermediate reasoning, or when they mutate the same conversation memory. When in doubt, model the shared state explicitly and pass it in as an immutable snapshot per branch.

PYTHON 3.11 - SUPERVISOR FAN-OUT

```

async def supervise(query: str) -> dict:
    async with asyncio.TaskGroup() as tg:
        market = tg.create_task(market_agent.invoke(query))
        compete = tg.create_task(competitor_agent.invoke(query))
        fin = tg.create_task(finance_agent.invoke(query))
    return await synth_agent.invoke({
        "market": market.result(),
        "competitor": compete.result(),
        "finance": fin.result(),
    })
  
```



FIGURE 3.3 Supervisor fan-out over specialist agents. Safe when subtasks are independent; hazardous the moment they share writable state.

3.4 Background & Fire-and-Forget

The last pattern is the one every team eventually needs and often gets wrong: work that must happen but must not block the response. Telemetry writes, cache warm-ups, memory-store updates, span exports — none of them should be on the critical path. The tool for the job is `asyncio.create_task`, but with two rules attached.

First, **you must hold the reference**. Python’s garbage collector is entitled to reap a task if nothing keeps it alive, and it will, mid-flight, silently, in production. Store tasks in a module-level `set` and remove them on completion. Second, **you must await them at shutdown**, or work you thought you’d persisted will vanish when the process exits.

PYTHON 3.11 - SAFE FIRE-AND-FORGET

```

bg_tasks: set[asyncio.Task] = set()

def fire_and_forget(coro):
    t = asyncio.create_task(coro)
    bg_tasks.add(t)
    t.add_done_callback(bg_tasks.discard) # auto-remove on finish

async def handle_chat(msg):
    reply = await agent.invoke(msg)
    fire_and_forget(log telemetry(msg, reply)) # non-blocking
    fire_and_forget(write_memory(msg, reply))
    return reply

async def shutdown():
    await asyncio.gather(*bg_tasks, return_exceptions=True)
  
```

⚠ GOTCHA

The single-line footgun is `asyncio.create_task(log(...))` without keeping the return value. It looks fine, ships fine, passes tests, then loses roughly a third of your telemetry under load. Always `bg_tasks.add(...)`.

CHAPTER FOUR

Concurrency Control — The Part Everyone Gets Wrong

Async without limits is a denial-of-service tool pointed at your own vendors. Structured concurrency is the fix.

Asynco makes it disarmingly easy to fire off a hundred concurrent LLM calls, which is exactly why most teams do — until the provider replies with a wall of 429s, backs off aggressively, and the p95 for every user in the system doubles for the next ten minutes. Concurrency is a resource; it needs an accountant.

Capping with a Semaphore

The simplest accountant is `asyncio.Semaphore`. It bounds the number of coroutines that can be in the critical section — usually “in-flight LLM call” — at any given time. Set it based on your provider’s actual rate limit, not on optimism.

PYTHON 3.11 - SEMAPHORE-BOUNDED FAN-OUT

```
SEM = asyncio.Semaphore(8) # at most 8 LLM calls at once

async def bounded_llm(prompt: str) -> str:
    async with SEM:
        return await llm_call(prompt)

async def batch(prompts):
    return await asyncio.gather(*(bounded_llm(p) for p in prompts))
```

Timeouts

Every external call gets a timeout. Not “usually” — every one. The 3.11 `asyncio.timeout` context manager is the modern form and composes cleanly; `asyncio.wait_for` still works but wraps the coroutine in a task and adds cancellation semantics that are surprising when nested.

15

PYTHON 3.11 - ASYNCIO.TIMEOUT VS WAIT_FOR

```
async with asyncio.timeout(10): # preferred, 3.11+
    result = await llm_call(prompt)

# legacy equivalent - still fine, less composable:
result = await asyncio.wait_for(llm_call(prompt), timeout=10)
```

Retry with exponential backoff

`tenacity` is the pragmatic choice. Wrap the leaf call — never the whole agent step — with a small retry policy that respects the provider’s semantics.

PYTHON 3.11 - TENACITY ASYNCRETRYING

```
from tenacity import AsyncRetrying, stop_after_attempt, wait_exponential, retry_if_exception_type
import httpx

async def robust_call(url: str) -> dict:
    async for attempt in AsyncRetrying(
        stop_after_attempt(5),
        wait=wait_exponential(multiplier=0.5, max=1),
        retry=retry_if_exception_type((httpx.HTTPError, asyncio.TimeoutError)),
        reraise=True,
    ):
        with attempt:
            async with httpx.AsyncClient(timeout=10) as c:
                r = await c.get(url)
                r.raise_for_status()
                return r.json()
```

Cancellation

A user closes the tab. FastAPI cancels the request. That cancellation must propagate all the way down to the in-flight LLM call, the vector search, and any tool executor — otherwise you burn provider quota generating a response nobody will read. In asyncio, cancellation raises `asyncio.CancelledError` at the next await; your job is to *not swallow it* and to run cleanup in `finally`.

16

PYTHON 3.11 - CANCEL-SAFE TOOL CALL

```
async def tool_call(payload: dict) -> dict:
    client = httpx.AsyncClient(timeout=10)
    try:
        r = await client.post("https://api/tool", json=payload)
        r.raise_for_status()
        return r.json()
    except asyncio.CancelledError:
        # user aborted - release resources, then re-raise
        raise
    finally:
        await client.aclose()
```

Structured concurrency with TaskGroup

The strongest reason to prefer `TaskGroup` over raw `gather` for agent orchestration is that `TaskGroup` gives you *structured* concurrency: if any child task raises, all siblings are cancelled, the group re-raises via `ExceptionGroup`, and cleanup runs deterministically. With `gather(..., return_exceptions=True)`, a raised exception in one branch leaves the sibling tasks running in the background as orphans — they will complete, log their own errors, and generally muddy your telemetry. With `return_exceptions=True` you avoid the orphan problem but you must remember to filter results yourself.

PYTHON 3.11 - TASKGROUP FOR ORCHESTRATION

```
async def step():
    try:
        async with asyncio.TaskGroup() as tg:
            a = tg.create_task(tool_a())
            b = tg.create_task(tool_b())
            c = tg.create_task(tool_c())
    except ExceptionGroup as eg:
        # eg.exceptions carries every child failure
        for err in eg.exceptions:
            logger.warning("tool failed: %s", err)
        raise
```

17

⚠️ GOTCHA — ONE BAD TOOL CRASHES THE WHOLE GATHER

With `asyncio.gather(*coros)` and default settings, the first exception bubbles up and the remaining coroutines keep running in the background — you get the exception AND the orphaned work. Teams frequently “fix” this by adding `return_exceptions=True`, which silences the error and lets a broken tool return exceptions-as-values that get fed to the next LLM step, producing hallucinated recovery. `TaskGroup` forces you to acknowledge the failure explicitly through `ExceptionGroup` — that’s the point.

17

ASYNC IN AGENTIC AI

CHAPTER FIVE SYNC VS ASYNC IN THE REAL FRAMEWORKS

CHAPTER FIVE

Sync vs Async in the Real Frameworks

You will not build agent infrastructure from asyncio primitives alone. Here’s the async surface of the five SDKs you will actually use.

LangChain / LangGraph

Every runnable in LangChain exposes both sync (`invoke`, `stream`, `batch`) and async (`ainvoke`, `astream`, `astream_events`, `abatch`) methods. In LangGraph, nodes are just runnables — inside an async graph, always call `ainvoke`. Mixing `invoke` and `ainvoke` in the same graph triggers a silent thread-pool fallback that we return to in the next chapter.

PYTHON 3.11 - LANGGRAPH ASYNC NODE

```
from langgraph.graph import StateGraph
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o")

async def plan_node(state):
    msg = await llm.ainvoke(state["messages"])
    return {"plan": msg.content}

graph = StateGraph(dict)
graph.add_node("plan", plan_node)
```

OpenAI SDK

The v1 Python SDK ships `AsyncOpenAI` alongside `openai`. Streaming chat completions become async iterators; use `async` for directly on the stream object.

18

ASYNC IN AGENTIC AI

CHAPTER FIVE SYNC VS ASYNC IN THE REAL FRAMEWORKS

PYTHON 3.11 - ASYNCOOPENAI STREAMING

```
from openai import AsyncOpenAI
client = AsyncOpenAI()

async def stream(prompt):
    stream = await client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": prompt}],
        stream=True,
    )
    async for chunk in stream:
        if (delta := chunk.choices[0].delta.content):
            yield delta
```

Anthropic SDK

Anthropic exposes `AsyncAnthropic` with a context-manager streaming API. Because it’s an async context manager, cancellation cleans up cleanly.

PYTHON 3.11 - ASYNCRANTHROPIC STREAMING

```
from anthropic import AsyncAnthropic
client = AsyncAnthropic()

async def stream(prompt):
    async with client.messages.stream(
        model="claude-sonnet-4",
        max_tokens=8192,
        messages=[{"role": "user", "content": prompt}],
    ) as s:
        async for text in s.text_stream:
            yield text
```

Databricks: Model Serving & Vector Search

On Databricks, an agent running inside Model Serving typically reaches other Model Serving endpoints and Vector Search indexes as its two dominant external calls. The MLflow deployments client is convenient but is synchronous under the hood; for hot paths, prefer `httpx.AsyncClient` against the endpoint URL directly, and use the async surface of Vector Search when available. See the [Databricks Agent Framework docs](#) for the current recommended patterns, and [databrickslabs](#) for reference implementations.

19

ASYNC IN AGENTIC AI

CHAPTER FIVE SYNC VS ASYNC IN THE REAL FRAMEWORKS

PYTHON 3.11 - DATABRICKS MODEL SERVING VIA HTTPX

```
import os, httpx

DB_HOST = os.environ["DATABRICKS_HOST"]
DB_TOKEN = os.environ["DATABRICKS_TOKEN"]

async def call_endpoint(name: str, payload: dict) -> dict:
    url = f"{DB_HOST}/serving-endpoints/{name}/invocations"
    hdr = {"Authorization": f"Bearer {DB_TOKEN}"}
    async with httpx.AsyncClient(timeout=10) as c:
        r = await c.post(url, headers=hdr, json=payload)
        r.raise_for_status()
        return r.json()
```

PYTHON 3.11 - VECTOR SEARCH (OFFLOADED WHEN THE CLIENT IS SYNC)

```
from databricks.vector_search.client import VectorSearchClient
import asyncio

vsc = VectorSearchClient()
index = vsc.get_index(endpoint_name="vs-prod", index_name="cat-sch-ab_ida")

async def retrieve(q: str, k: int = 6) -> list:
    # SDK is sync - offload to the loop keeps advancing
    return await asyncio.to_thread(
        index.similarity_search,
```


SDK	Sync API	Async API	Notes
LangChain / LangGraph	<code>invoke</code> , <code>stream</code> , <code>batch</code>	<code>ainvoke</code> , <code>astream</code> , <code>astream_events</code> , <code>abatch</code>	Mixing sync and async runnables triggers a thread-pool fallback — silent throughput cliff.
OpenAI	<code>OpenAI()</code>	<code>AsyncOpenAI()</code>	Streaming returns an async iterator, use <code>async for</code> .
Anthropic	<code>Anthropic()</code>	<code>AsyncAnthropic()</code>	<code>client.messages.stream(...)</code> is an async context manager — cancellation-clean.
Databricks Model Serving	MLflow deployments client	<code>httpx.AsyncClient</code> against the endpoint URL	Prefer direct <code>httpx</code> for hot paths; MLflow client is convenient but sync.
Databricks Vector Search	<code>VectorsearchClient</code>	Wrap with <code>asyncio.to_thread</code> today	Native async client is on the roadmap; offloading is the current safe pattern.

■ DATABRICKS / PRODUCTION NOTE

When your agent is deployed via the [Databricks Agent Framework](#), prefer its own async primitives and the framework's typed request/response contracts over hand-rolled HTTP. The framework wires tracing spans, MLflow evaluation, and Unity Catalog governance automatically; you inherit them for free by using its native async surface instead of raw `httpx`. The reference agents at [github.com/databrickslabs](#) are the source of truth for the current recommended shape.

CHAPTER SIX

Common Failure Modes with Fixes

Six anti-patterns that account for the vast majority of async-agent incidents. Print this page and pin it above the on-call desk.

Every one of these has shipped to production somewhere. Most of them do not manifest under a developer's local load and only surface once real traffic hits — which is why they show up in postmortems and not code reviews. Fix them structurally, not with a comment saying "todo".

Anti-pattern	Symptom	Fix
1. Blocking library in <code>async def</code> <code>requests.get</code> , <code>psycopg2</code> , <code>sync boto3</code> , <code>time.sleep</code>	Loop freezes for the full call duration. Concurrent requests queue up behind one blocked task. Latency looks fine at N=1, catastrophic at N=10.	Switch to async-native: <code>httpx</code> , <code>aioscraping</code> , <code>aiobotocore</code> , <code>asyncio.sleep</code> . As a bridge, wrap with <code>await asyncio.to_thread(fn, *args)</code> .
2. Await in a list comprehension <code>results = [await f(x) for x in xs]</code>	Serial execution masquerading as concurrent code. Total time = sum, not max. The word "async" appears in the source, but throughput is single-threaded.	<code>results = await asyncio.gather(*(f(x) for x in xs))</code> — or use a <code>TaskGroup</code> for structured concurrency.

<code>httpx.AsyncClient / db pool</code>	pool exhaustion, "Too many open files" a few hours after deploy. Restart papers over it.	<code>as_c: .</code> For long-lived clients, close explicitly in a FastAPI <code>lifespan</code> handler.
4. Mixing sync & async LangChain Calling <code>.invoke</code> inside an async node instead of <code>.ainvoke</code>	Silent thread-pool fallback: LangChain runs the sync call on the loop's default executor. Throughput plateaus at ~32 concurrent regardless of async-ness above.	Use <code>ainvoke / astream / abatch</code> consistently. Grep the codebase for <code>.invoke()</code> in async contexts.
5. Streaming without back-pressure	OOM on slow clients. Producer generates tokens faster than the network can flush them; unbounded buffer grows until the pod dies.	Insert a bounded <code>asyncio.Queue(maxsize=N)</code> between producer and consumer, or use an SSE library that honors back-pressure natively.
6. Sharing an <code>asyncio.Lock</code> across workers	Under multi-worker <code>gunicorn / uvicorn --workers N</code> , each worker has its own event loop. A "process-wide" lock protects nothing across workers; race conditions on shared state (cache, dedupe map).	Use per-worker locks for in-process state; use Redis / Postgres advisory locks / Unity Catalog transactions for cross-worker coordination.

⚠️ GOTCHA — THE BIGGEST ONE

Anti-pattern #4 is the single most damaging in real deployments because it looks correct in code review, passes tests, and only shows up as a puzzling throughput ceiling in production. When you migrate a LangChain agent to async, do a *whole-graph audit* for `.invoke` and non-`a-`prefixed calls. Add a lint rule if your team is large. The thread-pool fallback is fast enough that unit tests won't notice — but the default executor is 32 workers, so your ceiling is exactly 32 concurrent LLM calls per process, forever, regardless of how many `async def`s you sprinkle above it.

CHAPTER SEVEN

Production Architecture on Databricks

The end-to-end async stack that survives on-call rotation: FastAPI at the door, TaskGroup in the middle, Model Serving and Vector Search at the back.

Everything in the previous chapters composes into a single architecture, and it looks essentially the same across every serious deployment: an ASGI serving layer, an orchestrator that owns the agent step machine, and a set of async clients to the LLM, retrieval, tools and telemetry.

The serving layer: FastAPI + uvicorn (uvloop)

FastAPI + uvicorn with the uvloop event-loop policy is the default for a reason: uvloop is roughly 2–4× faster than the stdlib selector loop for the network-heavy workloads agents produce. Set it once at startup, the rest of the code stays vanilla asyncio.

PYTHON 3.11 - FASTAPI + STREAMING ENDPOINT

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.post("/chat")
async def chat(req: ChatRequest):
    async def gen():
        async for tok in agent.astream(req.messages):
            yield f"data: {tok}\n\n"
    return StreamingResponse(gen(), media_type="text/event-stream")

# Run with: uvicorn app:app --loop uvloop --http httptools --workers 4
```

The orchestrator: asyncio + TaskGroup

Between FastAPI and the outbound clients sits the orchestrator, which owns the ReAct loop, tool routing, and streaming state. TaskGroup gives it structured concurrency for tool fan-out; a module-level Semaphore protects the LLM endpoint; a per-request `asyncio.timeout` puts a hard ceiling on total wall time.

The clients: Model Serving, Vector Search.

Outbound, everything is async httpx to Model Serving (Chapter 5), Vector Search wrapped via `to_thread` until the native async client lands, and MLflow tracing for observability. MLflow's span API works inside async contexts as long as you use the context-manager form; spans nest correctly across `await` points.

PYTHON 3.11 · MLFLOW ASYNC TRACING

```
import mlflow

async def agent_step(state):
    with mlflow.start_span(name="plan") as span:
        plan = await llm.generate(state["message"])
        span.set_attribute("plan.tokens", len(plan.content))
        with mlflow.start_span(name="retrieve"):
            docs = await retrieve(plan.content)
    return {"plan": plan, "docs": docs}
```

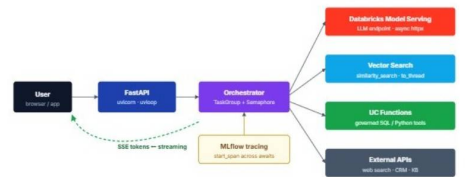


FIGURE 7.1 The reference architecture. Every arrow between orchestrator and back-end is an async client with a timeout and a semaphore; every span crosses `await` points cleanly.

■ DATABRICKS / PRODUCTION NOTE

Deploy the FastAPI app to Databricks Model Serving as a custom-agent endpoint, or wrap it in the [Databricks Agent Framework](#) and let Databricks handle serving, auth, and MLflow evaluation for you. Reference implementations at [github.com/databrickslabs](#) — in particular the `agent-framework` and `dox-instruct-cookbook` repos — show the current shape of production-grade async orchestrators against Unity Catalog and Vector Search.

CHAPTER EIGHT

Checklist — Is My Agent Truly Async?

Seven checks. If any box is unchecked, the agent isn't production-async — it is just an async-flavored sync agent, and it will fail the way one does.

- ☒ **No blocking calls in the hot path.**
Grep for `requests`, `time.sleep`, `sync psychopg2`, `sync boto3`, blocking file I/O. Replace with async-native clients or offload via `asyncio.to_thread`.
- ☒ **All LLM SDKs use the Async client.**
`AsyncOpenAI`, `AsyncAnthropic`, and consistently `aiinvoke` / `astream` for LangChain — no accidental `.invoke`.
- ☒ **Tool fan-out uses gather / TaskGroup with a Semaphore cap.**
Never unbounded. The semaphore size is set from the provider's actual rate limit, not from wishful thinking.
- ☒ **Every external call has a timeout.**
Preferably `asyncio.timeout(3.11)` at the orchestrator level, plus a per-client timeout on `httpx.AsyncClient`.
- ☒ **Streaming is end-to-end.**
LLM → orchestrator → HTTP response. No buffering the whole completion before returning it. First token to user in < 1s.
- ☒ **Cancellation is tested.**
Manually close a browser mid-stream and confirm the LLM request is actually cancelled upstream — not silently completed and discarded, burning quota.
- ☒ **Load-tested with concurrent users.**
`locust`, `vegeta`, or `ks`. Ramp until you find the ceiling; verify it's the provider or endpoint, not your own event loop.



If all seven boxes are checked, you have an agent whose latency floor is dominated by the model, not by your plumbing. That's the goal. Everything else — cost, throughput per GPU-hour, tail latency — falls out of doing this well.

REFERENCES

Further Reading

The canonical sources behind the patterns in this handbook. Bookmark them; the APIs move.

Azure Databricks — Generative AI Agent Framework

Reference architecture, deployment patterns, and MLflow-integrated evaluation for production agents on Databricks.

learn.microsoft.com/en-us/azure/databricks/generative-ai/agent-framework/

databrickslabs on GitHub

Reference implementations, community-driven tooling, and cookbook repos for agent-framework, DBRX and Vector Search.

[github.com/databrickslabs](https://github.com/databricks/databrickslabs)

Python `asyncio` — official documentation

Event loop, coroutines, Tasks, Futures, streams, subprocesses, synchronization primitives.

docs.python.org/3/library/asyncio.html

PEP 654 — Exception Groups and TaskGroup

The rationale for structured concurrency in the standard library.

peps.python.org/pep-0654/

tenacity

Retrying library with `AsyncRetrying` support — the pragmatic choice for exponential backoff in async code.

tenacity.readthedocs.io

httpx

Async HTTP client for Python — the recommended replacement for `requests` in any async code path.
www.python-httpx.org

FastAPI

ASGI framework used across every reference implementation in this handbook.
fastapi.tiangolo.com

uvloop

Drop-in replacement for the `asyncio` event loop; 2–4× faster on network-heavy workloads.
uvloop.readthedocs.io

— end of handbook —

Reference architecture, deployment patterns, and MLflow-integrated evaluation for production agents on Databricks.
learn.microsoft.com/en-us/azure/databricks/generative-ai/agent-framework/

databrickslabs on GitHub

Reference implementations, community-driven tooling, and cookbook repos for `agent-framework`, `DBRX` and `Vector Search`.
github.com/databrickslabs

Python `asyncio` — official documentation

Event loop, coroutines, `Tasks`, `Futures`, streams, subprocesses, synchronization primitives.
docs.python.org/3/library/asyncio.html

PEP 654 — Exception Groups and `TaskGroup`

The rationale for structured concurrency in the standard library.
peps.python.org/pep-0654/

tenacity

Retrying library with `AsyncRetrying` support — the pragmatic choice for exponential backoff in async code.
tenacity.readthedocs.io/en/latest/